

LLM from zero to Hero III

Optimizations Applied

Model Compilation

- Used `torch.compile()`
- Tested full model and granular compilation (decoder layers, attention modules)
- Observed minimal difference unless large subgraphs were compiled

Profiling

- Used `torch.profiler` to analyze inference/training
- Understood Self CPU/CUDA vs Total CPU/CUDA time
- Learned how kernels like `aten::addmm` and `volta_sgemv_*` map to operations

Manual Timing

- Created a `Timer` class with `cuda.synchronize()` for accurate CUDA timings
- Compared inference speed across compilation strategies

AMP (Mixed Precision Training)

- Integrated autocast and GradScaler
- Achieved **2–3× speedup per epoch**
- Verified numerical stability and memory savings
- On Single inference same performance in terms of speed.

I/O Bottleneck Check

- Measured raw `train_loader` time
- Found it to be ~4% of original epoch time — **not a bottleneck**

✓ Different levels of compilation

```
!wget https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt
```

```
--2025-08-06 11:39:02-- https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.111.133, 185.199.110.133, ..
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1115394 (1.1M) [text/plain]
Saving to: 'input.txt'
```

```
input.txt          100%[=====] 1.06M --.-KB/s in 0.02s
```

```
2025-08-06 11:39:02 (45.0 MB/s) - 'input.txt' saved [1115394/1115394]
```

```
import string
import torch
import math
import os
import numpy as np
import time
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset
from torch.utils.data import DataLoader
import torch.optim as optim
from torch.utils.tensorboard import SummaryWriter
from torch.optim.lr_scheduler import LambdaLR
from collections import Counter
from torch.profiler import profile, record_function, ProfilerActivity, schedule
import csv
import time
```

```
LOWER_CASE = True
seq_length = 64
batch_size = 256
embed_dim = 256
num_epochs = 4
```

```
num_heads = 4
num_att_layers = 2
```

```
f = open("input.txt")
data = f.read()
```

```
%load_ext tensorboard
```

▼ Functions and classes

```
def BPE_vocab(data):
    punctuation_set = set(string.punctuation)
    data_sep = word_separator(data=data.lower(), special_char = punctuation_set)
    data_word_char = [list(word) + ["</w>"] for word in data_sep]
    vocab = Counter(tuple(word) for word in data_word_char)
    pairs = Counter()
    for word, freq in vocab.items():
        for i in range(len(word) - 1):
            pair = (word[i], word[i+1])
            pairs[pair] += freq
    vocab = [''.join(word) for word, _ in pairs.most_common(100)] + list(set(data.lower())) + ["</w>"]
    return vocab
```

```
def BPE_enc(data, vocab):
    i = 0
    data_token = []
    while i < len(data):
        if i + 2 <= len(data) and data[i:i+2] in vocab:
            token = data[i:i+2]
            if token == '\n' and data_token and data_token[-1] == '\n':
                pass
            else:
                data_token.append(token)
            i += 2
        else:
            token = data[i:i+1]
            if token == '\n' and data_token and data_token[-1] == '\n':
                pass
            else:
                data_token.append(token)
            i += 1
    return data_token
```

```
def word_separator(data: str, special_char: list[str]) -> list[str]:
    """
    Separate text to words for tokenization
    Args:
        data: text
        speacial_char: special chatacters to break word
    Return:
        list[str]: list of words and special characters
    """
    data_separated = []
    word = ""
    for char in data:
        if char == " " or char == "\n" or char == "\t":
            if word != "":
                data_separated.append(word)
            word = ""
        elif char in special_char:
            data_separated.append(word)
            data_separated.append(char)
        else:
            word += char
    return data_separated
```

```
from torch.utils.data import Dataset
```

```
class TokenDataset(Dataset):
    def __init__(self, tokens, seq_len):
        self.tokens = tokens      # list or tensor of token IDs
        self.seq_len = seq_len    # length of input sequence

    def __len__(self):
        return len(self.tokens) - self.seq_len

    def __getitem__(self, idx):
```

```

    x = self.tokens[idx : idx + self.seq_len]
    y = self.tokens[idx + 1 : idx + self.seq_len + 1]
    return x, y

class PositionalEncoding(nn.Module):
    def __init__(self, d_model, max_len=5000):
        super(PositionalEncoding, self).__init__()

        # Create matrix of shape (max_len, d_model)
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1) # (max_len, 1)
        div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model))

        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        pe = pe.unsqueeze(0)

        # Register as buffer so it's not a parameter, but saved with the model
        self.register_buffer('pe', pe)

    def forward(self, x):
        """
        x: (batch_size, seq_len, d_model)
        """
        seq_len = x.size(1)
        # Add positional encoding
        x = x + self.pe[:, :seq_len]
        return x

class Attention(torch.nn.Module):
    def __init__(self, embedding_dim, seq_length):
        super(Attention, self).__init__()
        self.softmax = nn.Softmax(-1)
        self.register_buffer('causal', torch.tril(torch.ones(seq_length, seq_length)))

    def forward(self, q, k, v):
        B, T, _ = q.shape
        x = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(q.size(-1))
        causal_mask = self.causal[:T, :T]
        x = x.masked_fill(causal_mask == 0, float('-inf'))
        x = self.softmax(x)
        x = torch.matmul(x, v)
        return x

class MultiHeadAttention(torch.nn.Module):
    def __init__(self, embedding_dim, num_heads, seq_length):
        super(MultiHeadAttention, self).__init__()
        assert embedding_dim % num_heads == 0, "embedding_dim must be divisible by num_heads"

        self.attention = Attention(embedding_dim, seq_length)
        self.num_heads = num_heads
        self.head_dim = embedding_dim // num_heads

        # Final projection after concatenating heads
        self.out_proj = nn.Linear(embedding_dim, embedding_dim)

        self.q_proj = nn.Linear(embedding_dim, embedding_dim)
        self.k_proj = nn.Linear(embedding_dim, embedding_dim)
        self.v_proj = nn.Linear(embedding_dim, embedding_dim)

    def forward(self, x):
        B, T, C = x.shape # batch_size, seq_length, embed_size

        q = self.q_proj(x)
        k = self.k_proj(x)
        v = self.v_proj(x)

        q = q.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
        k = k.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)
        v = v.view(B, T, self.num_heads, self.head_dim).transpose(1, 2)

        # We need to merge batch and heads to call your single-head Attention:
        q = q.reshape(B * self.num_heads, T, self.head_dim)
        k = k.reshape(B * self.num_heads, T, self.head_dim)
        v = v.reshape(B * self.num_heads, T, self.head_dim)

        out = self.attention(q, k, v)
        out = out.view(B, self.num_heads, T, self.head_dim).transpose(1, 2)

```

```

    out = out.reshape(B, T, C)

    return self.out_proj(out)

class DecoderLayer(torch.nn.Module):
    def __init__(self, embedding_dim, num_head, seq_length):
        super(DecoderLayer, self).__init__()

        self.ff_1 = nn.Linear(embedding_dim, 4*embedding_dim)
        self.ff_2 = nn.Linear(4*embedding_dim, embedding_dim)
        self.m_head_att = MultiHeadAttention(embedding_dim, num_head, seq_length)
        self.layer_norm_1 = nn.LayerNorm(embedding_dim)
        self.layer_norm_2 = nn.LayerNorm(embedding_dim)

        # Activation
        self.relu = nn.ReLU()

    def forward(self, x):
        x_1 = self.m_head_att(x)
        x_2 = self.layer_norm_1(x_1 + x)
        x_3 = self.ff_1(x_2)
        x_4 = self.relu(x_3)
        x_5 = self.ff_2(x_4)
        x_6 = self.layer_norm_2(x_5 + x_2)
        return x_6

class DecoderOnlySmall(torch.nn.Module):
    def __init__(self, vocab_len, embedding_dim, num_head, seq_length, att_layer):
        super(DecoderOnlySmall, self).__init__()

        self.embed = nn.Embedding(vocab_len, embedding_dim)
        self.linear = nn.Linear(embedding_dim, vocab_len)
        self.DecLayer = DecoderLayer(embedding_dim, num_head, seq_length)

        # Activation
        self.softmax = nn.Softmax(dim=-1)
        self.relu = nn.ReLU()

        # weight tying
        self.linear.weight = self.embed.weight

        # positional encoding
        self.pos_encoding = PositionalEncoding(embedding_dim, max_len=seq_length)

        # Attention laywrs
        self.layers = nn.ModuleList([DecoderLayer(embedding_dim, num_head, seq_length) for _ in range(att_layer)])

    def forward(self, x):
        x_embed = self.embed(x)
        x_embed = self.pos_encoding(x_embed)

        # Decoder layers
        for layer in self.layers:
            x_embed = layer(x_embed)

        x_2 = self.linear(x_embed)
        if self.training:
            return x_2
        else:
            x_3 = self.softmax(x_2)
            return x_3

def train_epoch(epoch_index, tb_writer, training_loader, loss_fn, model, optimizer, scheduler, device):
    running_loss = 0.
    total_batches = 0
    epoch_times = []
    epoch_start = time.time()
    for i, data in enumerate(training_loader):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)

        for param in model.parameters():
            param.grad = None
        outputs = model(inputs)
        logits = outputs.view(-1, outputs.size(-1))
        labels = labels.view(-1)
        loss = loss_fn(logits, labels)
        loss.backward()

    optimizer.step()

```

```

scheduler.step()

running_loss += loss.item()
total_batches += 1

if i % 100 == 99:
    avg_loss_so_far = running_loss / total_batches
    print(f' batch {i+1} loss (avg so far): {avg_loss_so_far:.4f}')
    tb_x = epoch_index * len(training_loader) + i + 1
    tb_writer.add_scalar('Loss/train', avg_loss_so_far, tb_x)

epoch_time = time.time() - epoch_start
mean_loss = running_loss / total_batches

print(f"\nEpoch {epoch_index + 1} finished | Loss={mean_loss:.4f} | Time={epoch_time:.2f}s")

return mean_loss, epoch_time

def val_epoch(val_loader, tb_writer, model, loss_fn, epoch, device):
    running_loss = 0.
    with torch.no_grad():
        for inputs, labels in val_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = loss_fn(outputs.view(-1, outputs.size(-1)), labels.view(-1))
            running_loss += loss.item()
    mean_val_loss = running_loss / len(val_loader)
    tb_writer.add_scalar('Loss/val', mean_val_loss, epoch)
    return mean_val_loss

def get_lr_scheduler(optimizer, warmup_steps, total_steps):
    def lr_lambda(current_step):
        if current_step < warmup_steps:
            # linear warmup from 0 to 1
            return float(current_step) / float(max(1, warmup_steps))
        # cosine decay after warmup
        progress = float(current_step - warmup_steps) / float(max(1, total_steps - warmup_steps))
        return 0.5 * (1.0 + math.cos(math.pi * progress))
    return LambdaLR(optimizer, lr_lambda)

def train_model(model,
                train_loader,
                val_loader,
                loss_fn,
                optimizer,
                scheduler,
                device,
                num_epochs,
                save_dir,
                tb_writer=None):

    best_val_loss = float("inf")
    epochs_times = []

    # Initial evaluation before training
    model.eval()
    with torch.no_grad():
        val_loss = val_epoch(
            val_loader=val_loader,
            tb_writer=tb_writer,
            model=model,
            loss_fn=loss_fn,
            epoch=0,
            device=device
        )
    print(f"Epoch 0: Val Loss {val_loss:.4f}")

    for epoch in range(num_epochs):
        model.train()
        train_loss, epoch_time = train_epoch(
            epoch_index=epoch,
            tb_writer=tb_writer,
            training_loader=train_loader,
            loss_fn=loss_fn,
            model=model,
            optimizer=optimizer,
            scheduler=scheduler,
            device=device
        )
        epochs_times.append(epoch_time)

```

```

model.eval()
with torch.no_grad():
    val_loss = val_epoch(
        val_loader=val_loader,
        tb_writer=tb_writer,
        model=model,
        loss_fn=loss_fn,
        epoch=epoch,
        device=device
    )

print(f"Epoch {epoch + 1}: Train Loss {train_loss:.4f}, Val Loss {val_loss:.4f}")

if val_loss < best_val_loss:
    best_val_loss = val_loss
    checkpoint_path = os.path.join(save_dir, f"best_model_val_{val_loss:.4f}.pt")
    torch.save({
        "epoch": epoch,
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "train_loss": train_loss,
        "val_loss": val_loss
    }, checkpoint_path)
    print(f"Best model saved at epoch {epoch + 1} with Val Loss {val_loss:.4f}")

return epochs_times, best_val_loss

class Timer:
    def __init__(self):
        self.start_time = None
    def __enter__(self):
        if torch.cuda.is_available():
            torch.cuda.synchronize()
        self.start_time = time.time()
    def __exit__(self, exc_type, exc_val, exc_tb):
        if torch.cuda.is_available():
            torch.cuda.synchronize()
        elapsed = time.time() - self.start_time
        print(f"Elapsed time: {elapsed:.6f} seconds")

```

✓ Data Preprocessing

```

word_token = False
char_token = False
BPE = True

if LOWER_CASE:
    data = data.lower()
if word_token:
    punctuation_set = set(string.punctuation) # special characters
    data_sep = word_separator(data=data, special_char = punctuation_set)
    vocab = set(data_sep)
if BPE:
    vocab = BPE_vocab(data)
if char_token:
    data_sep = list(data)
    vocab = set(data_sep)

# Vocabulary
word2index = {word: i for i, word in enumerate(sorted(vocab))}
index2word = {i: word for word, i in word2index.items()}

if BPE:
    data_sep = BPE_enc(data, vocab)

# Tokenization
data_token = []
data_token = [word2index[word] for word in data_sep]

# Data Splitting
train = data_token[:int(len(data_token)*0.80)]
val = data_token[int(len(data_token)*0.80):int(len(data_token)*0.90)]
test = data_token[int(len(data_token)*0.90):]

# Create datasets
train_dataset = TokenDataset(torch.tensor(train, dtype=torch.long), seq_length)
val_dataset = TokenDataset(torch.tensor(val, dtype=torch.long), seq_length)
test_dataset = TokenDataset(torch.tensor(test, dtype=torch.long), seq_length)

```



```
optimizer = torch.optim.AdamW(model.parameters(), lr=base_lr)
scheduler = get_lr_scheduler(optimizer, warmup_steps, total_steps)
```

```
epochs_times_nor, best_val_loss = train_model(model,
                                              train_loader,
                                              val_loader,
                                              loss_fn,
                                              optimizer,
                                              scheduler,
                                              device,
                                              num_epochs,
                                              save_dir,
                                              tb_writer)
```

```

batch 100 loss (avg so far): 1.6371
batch 200 loss (avg so far): 1.6305
batch 300 loss (avg so far): 1.6246
batch 400 loss (avg so far): 1.6199
batch 500 loss (avg so far): 1.6151
batch 600 loss (avg so far): 1.6113
batch 700 loss (avg so far): 1.6069
batch 800 loss (avg so far): 1.6022
batch 900 loss (avg so far): 1.5979
batch 1000 loss (avg so far): 1.5937
batch 1100 loss (avg so far): 1.5892
batch 1200 loss (avg so far): 1.5850
batch 1300 loss (avg so far): 1.5807
batch 1400 loss (avg so far): 1.5764
batch 1500 loss (avg so far): 1.5723
batch 1600 loss (avg so far): 1.5680
batch 1700 loss (avg so far): 1.5641
batch 1800 loss (avg so far): 1.5601
batch 1900 loss (avg so far): 1.5560
batch 2000 loss (avg so far): 1.5521
batch 2100 loss (avg so far): 1.5480
batch 2200 loss (avg so far): 1.5439
batch 2300 loss (avg so far): 1.5399
batch 2400 loss (avg so far): 1.5359
batch 2500 loss (avg so far): 1.5320

```

Epoch 3 finished | Loss=1.5297 | Time=171.71s
 Epoch 3: Train Loss 1.5297, Val Loss 4.5274
 Best model saved at epoch 3 with Val Loss 4.5274

```

batch 100 loss (avg so far): 1.4279
batch 200 loss (avg so far): 1.4241
batch 300 loss (avg so far): 1.4202
batch 400 loss (avg so far): 1.4171
batch 500 loss (avg so far): 1.4137
batch 600 loss (avg so far): 1.4108
batch 700 loss (avg so far): 1.4084
batch 800 loss (avg so far): 1.4058
batch 900 loss (avg so far): 1.4033
batch 1000 loss (avg so far): 1.4011
batch 1100 loss (avg so far): 1.3988
batch 1200 loss (avg so far): 1.3967
batch 1300 loss (avg so far): 1.3945
batch 1400 loss (avg so far): 1.3925
batch 1500 loss (avg so far): 1.3908
batch 1600 loss (avg so far): 1.3891
batch 1700 loss (avg so far): 1.3874
batch 1800 loss (avg so far): 1.3858
batch 1900 loss (avg so far): 1.3843
batch 2000 loss (avg so far): 1.3830
batch 2100 loss (avg so far): 1.3817
batch 2200 loss (avg so far): 1.3805
batch 2300 loss (avg so far): 1.3793
batch 2400 loss (avg so far): 1.3783
batch 2500 loss (avg so far): 1.3774

```

Epoch 4 finished | Loss=1.3768 | Time=170.91s
 Epoch 4: Train Loss 1.3768, Val Loss 4.5238
 Best model saved at epoch 4 with Val Loss 4.5238

```

tb_writer.add_hparams(
{
    "batch_size": batch_size,
    "seq_length": seq_length,
    "lr": 0.001,
    "embed_dim": embed_dim,
    "num_layers": num_att_layers,
    "vocab_size": len(vocab)
},
{
    "best_val_loss": best_val_loss
}
)

```

▼ Compiled reduce-overhead

```
custom_run_name = f"comp_over_run_bs{batch_size}_seq{seq_length}_embed{embed_dim}_{int(time.time())}"
tb_writer = SummaryWriter(log_dir=f"runs/{custom_run_name}")
```

```
tinymodel = DecoderOnlySmall(len(vocab), embed_dim, num_heads, seq_length, num_att_layers)
tinymodel_compiled = torch.compile(tinymodel, mode="reduce-overhead")
model_over = tinymodel_compiled
```

```
total_steps = len(train_loader) * num_epochs
model_over.to(device)
loss_fn = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model_over.parameters(), lr=base_lr)
scheduler = get_lr_scheduler(optimizer, warmup_steps, total_steps)
```

```
epochs_times_over_comp, best_val_loss = train_model(model_over,
                                                    train_loader,
                                                    val_loader,
                                                    loss_fn,
                                                    optimizer,
                                                    scheduler,
                                                    device,
                                                    num_epochs,
                                                    save_dir,
                                                    tb_writer)
```



best model saved at epoch 4 with val loss 4.5238

```
tb_writer.add_hparams(
    {
        "batch_size": batch_size,
        "seq_length": seq_length,
        "lr": 0.001,
        "embed_dim": embed_dim,
        "num_layers": num_att_layers,
        "vocab_size": len(vocab)
    },
    {
        "best_val_loss": best_val_loss
    }
)
```

✓ Compile Decoder layers

```
custom_run_name = f"dec_comp_bs{batch_size}_seq{seq_length}_embed{embed_dim}_{int(time.time())}"
tb_writer = SummaryWriter(log_dir=f"runs/{custom_run_name}")
```

```
tinymodel = DecoderOnlySmall(len(vocab), embed_dim, num_heads, seq_length, num_att_layers)
tinymodel.layers = nn.ModuleList([torch.compile(layer) for layer in tinymodel.layers])
model_dec_comp = tinymodel
```

```
total_steps = len(train_loader) * num_epochs
model_dec_comp.to(device)
loss_fn = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model_dec_comp.parameters(), lr=base_lr)
scheduler = get_lr_scheduler(optimizer, warmup_steps, total_steps)
```

```
epochs_times_over_dec, best_val_loss = train_model(model_dec_comp,
                                                    train_loader,
                                                    val_loader,
                                                    loss_fn,
                                                    optimizer,
                                                    scheduler,
                                                    device,
                                                    num_epochs,
                                                    save_dir,
                                                    tb_writer)
```



```

batch 1200 loss (avg so far): 1.4353
batch 1300 loss (avg so far): 1.4514
batch 1400 loss (avg so far): 1.4496
batch 1500 loss (avg so far): 1.4480
batch 1600 loss (avg so far): 1.4464
batch 1700 loss (avg so far): 1.4448
batch 1800 loss (avg so far): 1.4433
batch 1900 loss (avg so far): 1.4419
batch 2000 loss (avg so far): 1.4406
batch 2100 loss (avg so far): 1.4395
batch 2200 loss (avg so far): 1.4384
batch 2300 loss (avg so far): 1.4373
batch 2400 loss (avg so far): 1.4364
batch 2500 loss (avg so far): 1.4355

```

```

Epoch 4 finished | Loss=1.4349 | Time=157.41s
Epoch 4: Train Loss 1.4349, Val Loss 4.5284
Best model saved at epoch 4 with Val Loss 4.5284

```

▼ Compile MHA layers

```

custom_run_name = f"mha_comp_bs{batch_size}_seq{seq_length}_embed{embed_dim}_{int(time.time())}"
tb_writer = SummaryWriter(log_dir=f"runs/{custom_run_name}")

tinymodel = DecoderOnlySmall(len(vocab), embed_dim, num_heads, seq_length, num_att_layers)
for layer in tinymodel.layers:
    layer.m_head_att = torch.compile(layer.m_head_att)
model_mha_comp = tinymodel

total_steps = len(train_loader) * num_epochs
model_mha_comp.to(device)
loss_fn = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model_mha_comp.parameters(), lr=base_lr)
scheduler = get_lr_scheduler(optimizer, warmup_steps, total_steps)

epochs_times_over_mha, best_val_loss = train_model(model_mha_comp,
                                                    train_loader,
                                                    val_loader,
                                                    loss_fn,
                                                    optimizer,
                                                    scheduler,
                                                    device,
                                                    num_epochs,
                                                    save_dir,
                                                    tb_writer)

```



```

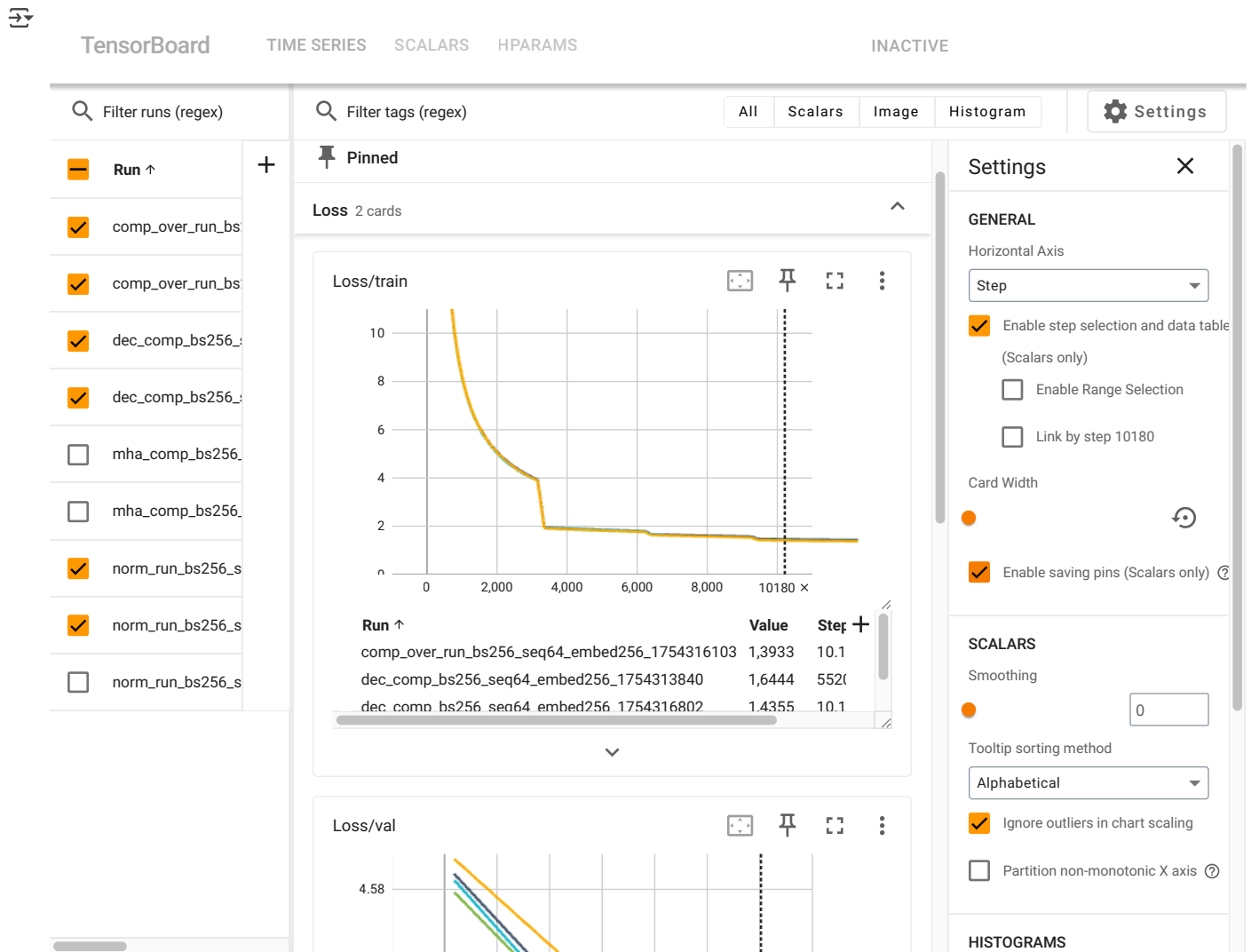
batch 1000 loss (avg so far): 1.4420
batch 1100 loss (avg so far): 1.4406
batch 1200 loss (avg so far): 1.4386
batch 1300 loss (avg so far): 1.4368
batch 1400 loss (avg so far): 1.4349
batch 1500 loss (avg so far): 1.4331
batch 1600 loss (avg so far): 1.4315
batch 1700 loss (avg so far): 1.4299
batch 1800 loss (avg so far): 1.4284
batch 1900 loss (avg so far): 1.4271
batch 2000 loss (avg so far): 1.4259
batch 2100 loss (avg so far): 1.4248
batch 2200 loss (avg so far): 1.4236
batch 2300 loss (avg so far): 1.4226
batch 2400 loss (avg so far): 1.4216
batch 2500 loss (avg so far): 1.4207

```

Epoch 4 finished | Loss=1.4202 | Time=163.34s
 Epoch 4: Train Loss 1.4202, Val Loss 4.5273
 Best model saved at epoch 4 with Val Loss 4.5273

▼ Compare

```
%tensorboard --logdir runs
```



```

model.eval()
with torch.no_grad():
    out = model(test_dataset.tokens[-64:].unsqueeze(0).to(device))
index2word[torch.argmax(out[:, -1, :]).item()]

```

```
→ 'k'
```

```

pred_length = 150
pred = []
generated_tokens = test_dataset.tokens[-64:].tolist()

```

```

for _ in range(pred_length):
    inp = torch.tensor(generated_tokens[-64:], device=device).unsqueeze(0)
    with torch.no_grad():

```

```

    out = model(inp)
    out_2 = model_over(inp)
    next_token_id = torch.argmax(out[:, -1, :], dim=-1).item()
    generated_tokens.append(next_token_id)
    pred.append(index2word[next_token_id])

```

```

pretty_text = ''.join(pred)
print(pretty_text)

```

```

→ king richard iii:
  why, then, i say; i will not be a but to them.
  good sirrah, i say! i will not be a but
  that i shall be the first tongues, and the prince,
  that the man of the king, and the king's,

```

✓ Performance

```

with Timer():
    with torch.no_grad():
        model(inp)
with Timer():
    with torch.no_grad():
        model_over(inp)
with Timer():
    with torch.no_grad():
        model_dec_comp(inp)
with Timer():
    with torch.no_grad():
        model_mha_comp(inp)

```

```

→ Elapsed time: 0.004099 seconds
  Elapsed time: 0.000976 seconds
  Elapsed time: 0.001458 seconds
  Elapsed time: 0.001550 seconds

```

```

activities = [ProfilerActivity.CPU]
if torch.cuda.is_available():
    device = "cuda"
    activities += [ProfilerActivity.CUDA]
elif torch.xpu.is_available():
    device = "xpu"
    activities += [ProfilerActivity.XPU]
else:
    print(
        "Neither CUDA nor XPU devices are available to demonstrate profiling on acceleration devices"
    )
    import sys

    sys.exit(0)

```

```

model.eval()
model_over.eval()
model_dec_comp.eval()
model_mha_comp.eval()
inputs = torch.tensor(test_dataset.tokens[-64:].tolist(), device=device).unsqueeze(0)

```

```

with profile(activities=activities) as prof:
    with torch.no_grad():
        model(inputs)
with profile(activities=activities) as prof_over:
    with torch.no_grad():
        model_over(inputs)
with profile(activities=activities) as prof_dec:
    with torch.no_grad():
        model_dec_comp(inputs)
with profile(activities=activities) as prof_mha:
    with torch.no_grad():
        model_mha_comp(inputs)

```

```

prof.export_chrome_trace("trace.json")
prof_over.export_chrome_trace("trace_2.json")
prof_dec.export_chrome_trace("trace_2.json")
prof_mha.export_chrome_trace("trace_2.json")
# chrome://tracing

```

```

sort_by_keyword = "self_" + device + "_time_total"
print(prof.key_averages().table(sort_by=sort_by_keyword, row_limit=10))
print(prof_over.key_averages().table(sort_by=sort_by_keyword, row_limit=10))

```

```
print(prof_dec.key_averages().table(sort_by=sort_by_keyword, row_limit=10))
print(prof_mha.key_averages().table(sort_by=sort_by_keyword, row_limit=10))
```

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU ti
aten::addmm	6.97%	765.998us	11.20%	1.231ms	94
volta_sgemmm_128x32_tn	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_32x32_sliced1x4_tn	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_64x32_sliced1x4_tn	0.00%	0.000us	0.00%	0.000us	0
aten::bmm	1.94%	212.926us	2.62%	287.429us	71
void cublasLt::splitKreduce_kernel<32, 16, int, floa...	0.00%	0.000us	0.00%	0.000us	0
aten::native_layer_norm	1.04%	114.052us	2.50%	274.357us	68
void at::native::(anonymous namespace)::vectorized_l...	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_32x128_tn	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_32x128_nn	0.00%	0.000us	0.00%	0.000us	0
Self CPU time total: 10.990ms					
Self CUDA time total: 548.760us					

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU ti
Torch-Compiled Region: 0/3	12.15%	456.368us	90.58%	3.401ms	3
volta_sgemmm_128x64_tn	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_128x32_tn	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_32x32_sliced1x4_tn	0.00%	0.000us	0.00%	0.000us	0
void cublasLt::splitKreduce_kernel<32, 16, int, floa...	0.00%	0.000us	0.00%	0.000us	0
void cublasLt::splitKreduce_kernel<32, 16, int, floa...	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_32x128_tn	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_32x128_nn	0.00%	0.000us	0.00%	0.000us	0
triton_per_fused_add_native_layer_norm_3	0.00%	0.000us	0.00%	0.000us	0
triton_per_fused__softmax_div_eq_masked_fill_1	0.00%	0.000us	0.00%	0.000us	0
Self CPU time total: 3.755ms					
Self CUDA time total: 510.865us					

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU ti
aten::mm	3.90%	216.822us	6.08%	337.800us	56
aten::addmm	5.65%	314.107us	10.06%	558.979us	79
volta_sgemmm_128x64_tn	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_128x32_tn	0.00%	0.000us	0.00%	0.000us	0
aten::bmm	1.94%	107.694us	2.69%	149.289us	37
volta_sgemmm_32x32_sliced1x4_tn	0.00%	0.000us	0.00%	0.000us	0
void cublasLt::splitKreduce_kernel<32, 16, int, floa...	0.00%	0.000us	0.00%	0.000us	0
void cublasLt::splitKreduce_kernel<32, 16, int, floa...	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_32x128_tn	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_32x128_nn	0.00%	0.000us	0.00%	0.000us	0
Self CPU time total: 5.555ms					
Self CUDA time total: 477.589us					

Name	Self CPU %	Self CPU	CPU total %	CPU total	CPU ti
aten::addmm	10.11%	630.178us	17.05%	1.062ms	81
volta_sgemmm_128x32_tn	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_32x32_sliced1x4_tn	0.00%	0.000us	0.00%	0.000us	0
volta_sgemmm_64x32_sliced1x4_tn	0.00%	0.000us	0.00%	0.000us	0

```
import statistics
summary_lines = [
    line for line in prof.key_averages().table(sort_by=sort_by_keyword, row_limit=10).splitlines() if line.startswith("Self
"]
```

```
summary_lines
```

```
['Self CPU time total: 10.990ms', 'Self CUDA time total: 548.760us']
```

```
epochs_times_nor
```

```
[172.33719944953918,
172.16205501556396,
171.71431279182434,
170.90737462043762]
```

```
epochs_times_over_comp
```

```
[174.8033685684204, 157.74860262870789, 156.78219318389893, 155.84034729003906]
```

```
epochs_times_over_dec
```

```
[160.16022658348083, 158.15562272071838, 156.4211781024933, 157.41248655319214]
```

epochs_times_over_mha

↻ [163.79189777374268, 162.15518379211426, 163.1937747001648, 163.3440387248993]

✓ Automatic mixed Precision

✓ Modified functions

```
def train_epoch(epoch_index, tb_writer, training_loader, loss_fn, model, optimizer, scheduler, device):
    running_loss = 0.
    total_batches = 0
    epoch_times = []
    epoch_start = time.time()
    for i, data in enumerate(training_loader):
        inputs, labels = data
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad(set_to_none=True)

        # Autocast
        with torch.autocast(device_type='cuda', dtype=torch.float16):
            outputs = model(inputs)
            logits = outputs.view(-1, outputs.size(-1))
            labels = labels.view(-1)
            loss = loss_fn(logits, labels)

        scheduler.step()
        scaler.scale(loss).backward()
        scaler.step(optimizer)
        scaler.update()

        running_loss += loss.item()
        total_batches += 1

    if i % 100 == 99:
        avg_loss_so_far = running_loss / total_batches
        print(f' batch {i+1} loss (avg so far): {avg_loss_so_far:.4f}')
        tb_x = epoch_index * len(training_loader) + i + 1
        tb_writer.add_scalar('Loss/train', avg_loss_so_far, tb_x)

    epoch_time = time.time() - epoch_start
    mean_loss = running_loss / total_batches

    print(f"\nEpoch {epoch_index + 1} finished | Loss={mean_loss:.4f} | Time={epoch_time:.2f}s")

    return mean_loss, epoch_time

def train_model(model,
                train_loader,
                val_loader,
                loss_fn,
                optimizer,
                scheduler,
                device,
                num_epochs,
                save_dir,
                tb_writer=None):

    best_val_loss = float("inf")
    epochs_times = []

    # Initial evaluation before training
    model.eval()
    with torch.no_grad():
        val_loss = val_epoch(
            val_loader=val_loader,
            tb_writer=tb_writer,
            model=model,
            loss_fn=loss_fn,
            epoch=0,
            device=device
        )
    print(f"Epoch 0: Val Loss {val_loss:.4f}")

    for epoch in range(num_epochs):
        model.train()
```

```

train_loss, epoch_time = train_epoch(
    epoch_index=epoch,
    tb_writer=tb_writer,
    training_loader=train_loader,
    loss_fn=loss_fn,
    model=model,
    optimizer=optimizer,
    scheduler=scheduler,
    device=device
)
epochs_times.append(epoch_time)

model.eval()
with torch.no_grad():
    val_loss = val_epoch(
        val_loader=val_loader,
        tb_writer=tb_writer,
        model=model,
        loss_fn=loss_fn,
        epoch=epoch,
        device=device
    )

print(f"Epoch {epoch + 1}: Train Loss {train_loss:.4f}, Val Loss {val_loss:.4f}")

if val_loss < best_val_loss:
    best_val_loss = val_loss
    checkpoint_path = os.path.join(save_dir, f"best_model_val_{val_loss:.4f}.pt")
    torch.save({
        "epoch": epoch,
        "model_state_dict": model.state_dict(),
        "optimizer_state_dict": optimizer.state_dict(),
        "train_loss": train_loss,
        "val_loss": val_loss
    }, checkpoint_path)
    print(f"Best model saved at epoch {epoch + 1} with Val Loss {val_loss:.4f}")

return epochs_times, best_val_loss

```

▼ Data Preprocessing

```

if LOWER_CASE:
    data = data.lower()
if word_token:
    punctuation_set = set(string.punctuation) # special characters
    data_sep = word_separator(data=data, special_char = punctuation_set)
    vocab = set(data_sep)
if BPE:
    vocab = BPE_vocab(data)
if char_token:
    data_sep = list(data)
    vocab = set(data_sep)

# Vocabulary
word2index = {word: i for i, word in enumerate(sorted(vocab))}
index2word = {i: word for word, i in word2index.items()}

if BPE:
    data_sep = BPE_enc(data, vocab)

# Tokenization
data_token = []
data_token = [word2index[word] for word in data_sep]

# Data Splitting
train = data_token[:int(len(data_token)*0.80)]
val = data_token[int(len(data_token)*0.80):int(len(data_token)*0.90)]
test = data_token[int(len(data_token)*0.90):]

# Create datasets
train_dataset = TokenDataset(torch.tensor(train, dtype=torch.long), seq_length)
val_dataset = TokenDataset(torch.tensor(val, dtype=torch.long), seq_length)
test_dataset = TokenDataset(torch.tensor(test, dtype=torch.long), seq_length)

# Wrap in dataloaders
train_loader= DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=2, pin_memory=True, prefetch_factor=2)
val_loader= DataLoader(val_dataset, batch_size=batch_size, shuffle=False, num_workers=2, pin_memory=True, prefetch_factor=2)
test_loader= DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=2, pin_memory=True, prefetch_factor=2)

```

✓ Model training

```
compile = True
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
base_lr = 0.001
warmup_steps = 100
save_dir = "checkpoints"
os.makedirs(save_dir, exist_ok=True)
os.makedirs("log_profiler", exist_ok=True)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

custom_run_name = f"AMP_run_bs{batch_size}_seq{seq_length}_embed{embed_dim}_{int(time.time())}"
tb_writer = SummaryWriter(log_dir=f"runs/{custom_run_name}")

tinymodel = DecoderOnlySmall(len(vocab), embed_dim, num_heads, seq_length, num_att_layers)
tinymodel_compiled = torch.compile(tinymodel, mode="reduce-overhead")
model_amp = tinymodel_compiled

total_steps = len(train_loader) * num_epochs
model_amp.to(device)
loss_fn = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model_amp.parameters(), lr=base_lr)
scaler = torch.amp.GradScaler()
scheduler = get_lr_scheduler(optimizer, warmup_steps, total_steps)

epochs_times_amp, best_val_loss = train_model(model_amp,
                                              train_loader,
                                              val_loader,
                                              loss_fn,
                                              optimizer,
                                              scheduler,
                                              device,
                                              num_epochs,
                                              save_dir,
                                              tb_writer)
```



```

batch 1700 loss (avg so far): 1.4342
batch 1800 loss (avg so far): 1.4327
batch 1900 loss (avg so far): 1.4315
batch 2000 loss (avg so far): 1.4302
batch 2100 loss (avg so far): 1.4290
batch 2200 loss (avg so far): 1.4278
batch 2300 loss (avg so far): 1.4266
batch 2400 loss (avg so far): 1.4258
batch 2500 loss (avg so far): 1.4249

```

```

Epoch 4 finished | Loss=1.4244 | Time=49.28s
Epoch 4: Train Loss 1.4244, Val Loss 4.5273
Best model saved at epoch 4 with Val Loss 4.5273

```

epochs_times_amp

```
↗ [73.52976059913635, 50.556657552719116, 49.60465908050537, 49.27888774871826]
```

```
inputs = torch.tensor(test_dataset.tokens[-64:].tolist(), device=device).unsqueeze(0)
```

```

with Timer():
    with torch.no_grad():
        with torch.autocast(device_type='cuda', dtype=torch.float16):
            output = model_amp(inputs)

```

```
↗ Elapsed time: 0.001025 seconds
```

✓ IO bottleneck

```

start = time.time()
for batch in train_loader:
    pass
print("Data loading time:", time.time() - start)

```

```
↗ Data loading time: 7.5734992027282715
```

Empieza a programar o a [crear código](#) con IA.